Experiment No. 4

Aim: Implement K-Nearest Neighbors (KNN) and evaluate model performance

Objectives:

1. To understand the working principle of the K-Nearest Neighbors (KNN) algorithm for classification tasks.
2. To preprocess the dataset by performing feature selection, train–test splitting, and feature scaling.
3. To implement the KNN classifier using Python's scikit-learn library.

Theory:

The implementation of the **K-Nearest Neighbors (KNN)** algorithm generally follows three main stages:
data preparation, **model creation**, and **model evaluation**.
Python's **scikit-learn** library is the standard tool for this workflow.

Sample Dataset (Used for KNN Classification):

For demonstrating the K-Nearest Neighbors (KNN) algorithm, a **synthetic student performance dataset** is used.
Each record represents a student, described using academic and behavioral features.
The target variable indicates whether the student **passes (1)** or **fails (0)** based on predefined conditions.

Features Description

Feature Name	Description
Study Hours	Average number of hours studied per day
Attendance (%)	Percentage of classes attended
Internal Score	Average internal assessment marks
Sleep Hours	Average hours of sleep per day
Previous GPA	GPA obtained in the previous semester
Result (Target)	1 = Pass, 0 = Fail

Sample Data Table

Study Hours	Attendance (%)	Internal Score	Sleep Hours	Previous GPA	Result
6.2	82	68	7.5	7.2	1
3.8	65	50	6.0	5.8	0

Target Variable Logic:

The **Result** column is generated using the following criteria:

- Study Hours > 4
- Attendance > 70%
- Internal Score > 55
- Previous GPA > 6

If all conditions are satisfied, the student is labeled Pass (1); otherwise Fail (0).

Q. Why Is This Dataset Suitable for KNN?

- Contains **multiple numerical features**, ideal for distance-based learning
- Requires **feature scaling**, demonstrating the importance of preprocessing
- Binary classification problem, perfect for **KNN classifier**
- Mimics a **real-world academic performance scenario**

---

1. Prepare and Preprocess the Data

1.1 Load the Dataset

- Load your dataset into a suitable structure, typically a **pandas DataFrame**.

1.2 Split the Data

- Separate the dataset into:
  - **Features (X)**: input variables
  - **Labels (y)**: target variable
- Split the data into **training** and **testing** sets using `train_test_split`.

1.3 Feature Scaling

- **Essential for KNN**, since it is a distance-based algorithm.
- Features with larger numeric ranges can dominate distance calculations.
- Apply normalization or standardization using tools like:
    - `StandardScaler` (most common)
    - `MinMaxScaler` (optional alternative)

2. Implement the KNN Model

2.1 Initialize the Model

- Choose the appropriate KNN class:
    - **Classification** → `KNeighborsClassifier`
    - **Regression** → `KNeighborsRegressor`
- Specify the number of neighbors:
    - `n_neighbors = k`

2.2 Train the Model

- Fit the model to the training data using the `fit()` method.

2.3 Make Predictions

- Use the `predict()` method to generate predictions on the test dataset.

3. Evaluate Model Performance

Evaluating performance involves comparing **predicted values** with **actual test values**.

3.1 Choose Evaluation Metrics

For Classification Tasks:

- **Accuracy**
    - Proportion of correct predictions.
- **Confusion Matrix**
    - Displays:
        - True Positives (TP)
        - True Negatives (TN)
        - False Positives (FP)
        - False Negatives (FN)
- **Precision, Recall, and F1-Score**

    - Especially useful for **imbalanced datasets**.
    - Provide class-level performance insights.

For Regression Tasks:

- **Mean Squared Error (MSE)**
    - Measures the average squared difference between predicted and actual values.

3.2 Compute the Metrics

- Use built-in scikit-learn functions such as:
    - `accuracy_score`
    - `confusion_matrix`
    - `classification_report`
    - `mean_squared_error`

4. Tuning the k Hyperparameter

The value of **k** directly affects the **bias–variance trade-off**.

4.1 Impact of k Values

- **Small k**
    - Highly sensitive to noise and outliers
    - Higher risk of **overfitting**
- **Large k**
    - Produces smoother decision boundaries
    - Higher risk of **underfitting**

4.2 Finding the Optimal k

- Use techniques such as:
    - **Cross-validation** (e.g., k-fold cross-validation)
    - Plotting **error rate or accuracy vs. k**
- Select the k value with the best performance
    - Often identified using an **elbow method** visualization

4.3 Tip for Binary Classification

- Use an **odd value of k** to avoid ties in majority voting.

Code:

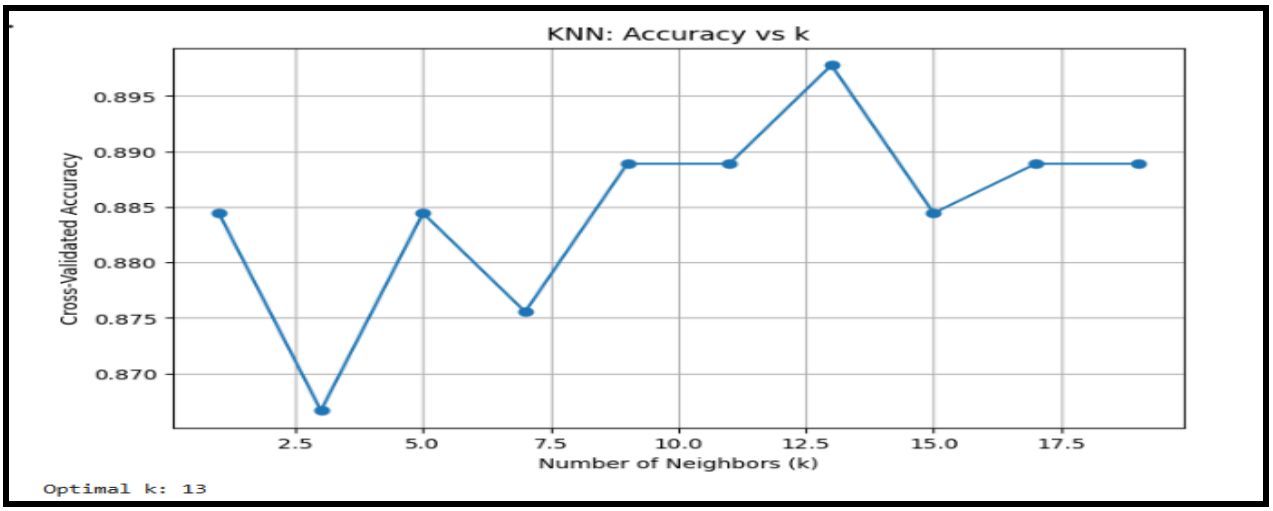
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

np.random.seed(42)
data_size = 300
data = {
    "study_hours": np.random.normal(5, 2, data_size),
    "attendance": np.random.normal(75, 10, data_size),
    "internal_score": np.random.normal(60, 15, data_size),
    "sleep_hours": np.random.normal(7, 1.5, data_size),
    "previous_gpa": np.random.normal(6.5, 1, data_size)
}

df = pd.DataFrame(data)
# Create target variable
df["result"] = (
    (df["study_hours"] > 4) &
    (df["attendance"] > 70) &
    (df["internal_score"] > 55) &
    (df["previous_gpa"] > 6)
).astype(int)
df.head()
X = df.drop("result", axis=1)
y = df["result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

k = 5  # odd value to avoid tie
```

```
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)
k_values = range(1, 21, 2)
accuracy_scores = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train_scaled, y_train, cv=5)
    accuracy_scores.append(scores.mean())
plt.figure(figsize=(8,5))
plt.plot(k_values, accuracy_scores, marker='o')
plt.xlabel("Number of Neighbors (k)")
plt.ylabel("Cross-Validated Accuracy")
plt.title("KNN: Accuracy vs k")
plt.grid(True)
plt.show()
optimal_k = k_values[np.argmax(accuracy_scores)]
print("Optimal k:", optimal_k)
```

Output:

	study_hours	attendance	internal_score	sleep_hours	previous_gpa	result
0	5.993428	66.710050	71.354829	7.553010	6.625225	0
1	4.723471	69.398190	46.167520	6.409992	6.070594	0
2	6.295377	82.472936	73.044089	7.043117	6.622298	1
3	8.046060	81.103703	80.334568	8.917678	7.043298	1
4	4.531693	74.790984	66.201524	7.286649	6.548860	1

```
KNeighborsClassifier
KNeighborsClassifier()
```



The graph shows that cross-validated accuracy varies with the number of neighbors (*k*), reaching its highest value around k = 13, indicating the optimal choice of *k* for the KNN model.

Conclusion:

The K-Nearest Neighbors (KNN) algorithm was successfully implemented and evaluated for classifying student performance. Proper data preprocessing and feature scaling improved model accuracy. The experiment showed that model performance depends strongly on the choice of *k*, and selecting an optimal *k* using cross-validation enhanced generalization. Overall, KNN proved to be a simple and effective classification method.